

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 18 EXERCISES

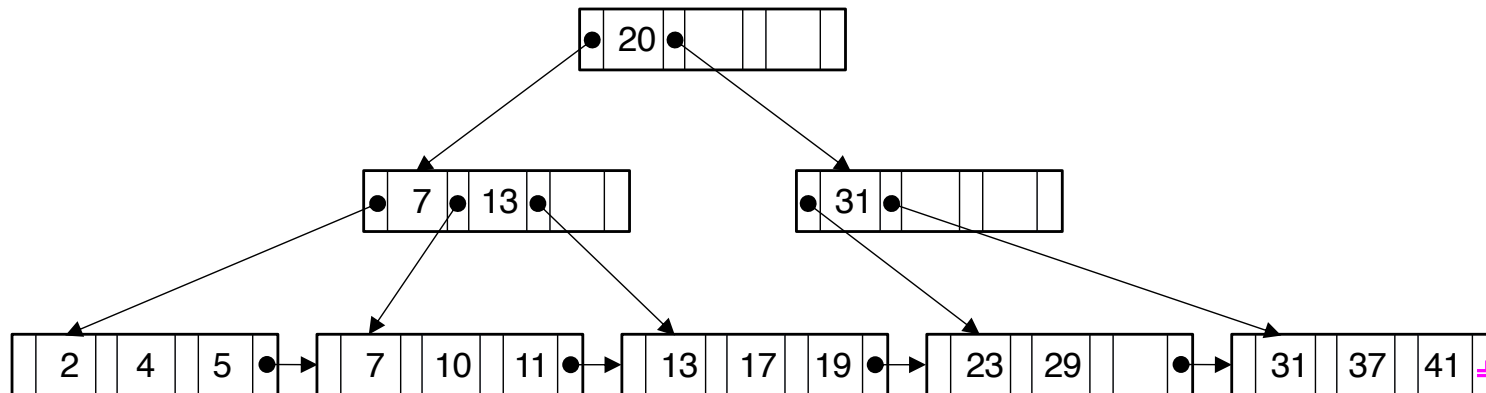
INDEXING:

B+-TREE INDEX AND HASH INDEX

EXERCISE I

Using initial the B⁺-tree given below with **fan out 4** and the B⁺-tree insertion and deletion algorithms given in the lecture notes, show the B⁺-tree that would result **after** performing the following operations **in the order given**.

- i. insert 3
- ii. insert 8



Non-leaf nodes: $\min \lceil 4/2 \rceil = 2$ pointers; $\min \lceil 4/2 \rceil - 1 = 1$ value

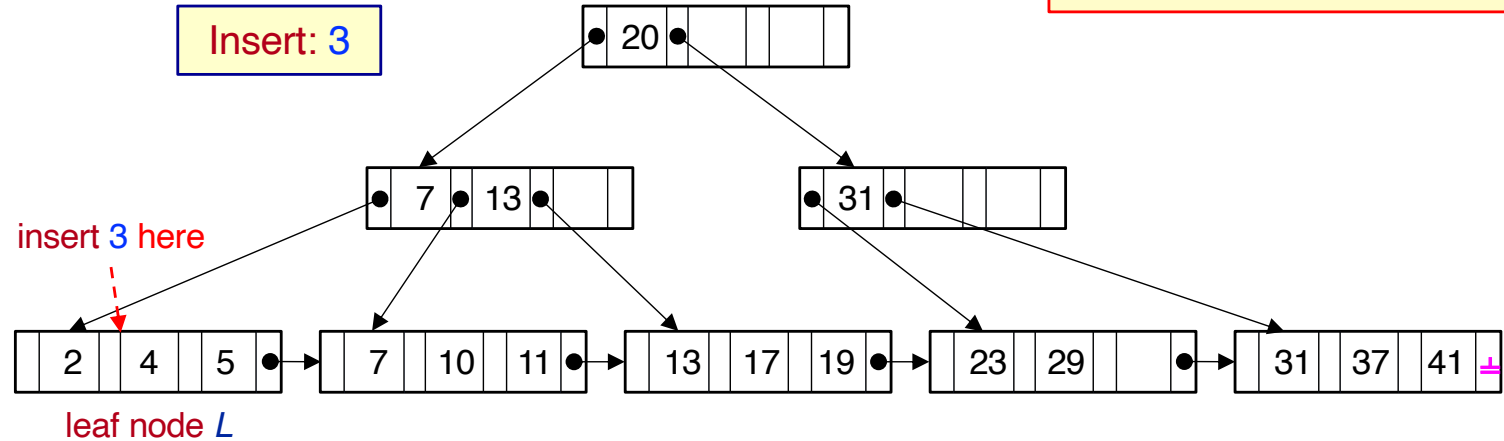
Leaf nodes: $\min \lceil (4-1)/2 \rceil + 1 = 3$ pointers; $\min \lceil (4-1)/2 \rceil = 2$ values

EXERCISE 1 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

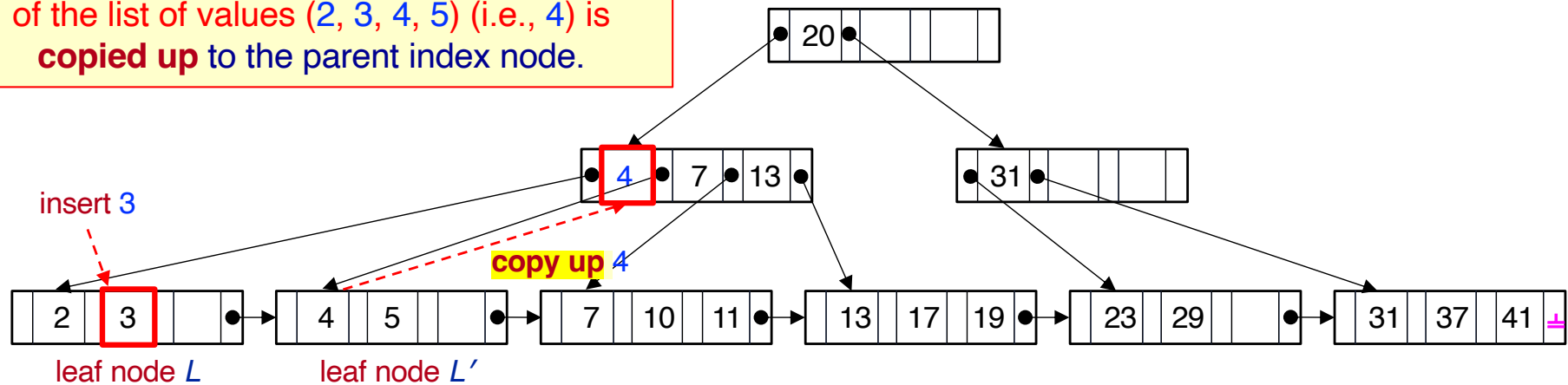
The insertion causes the first leaf node to become **overfull**.

Insert: 3



B⁺-tree before insertion of 3.

The leaf node is split and the search-key value of the node entry at position $\lceil n/2 \rceil + 1 = 3$ of the list of values (2, 3, 4, 5) (i.e., 4) is **copied up** to the parent index node.



B⁺-tree after insertion of 3.

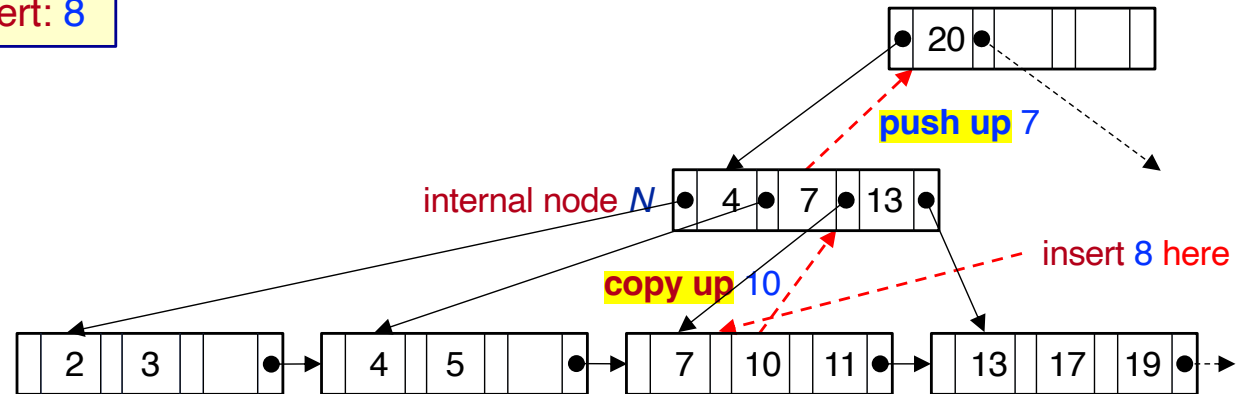
EXERCISE 1 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

The insertion causes the third leaf node to become **overfull**.

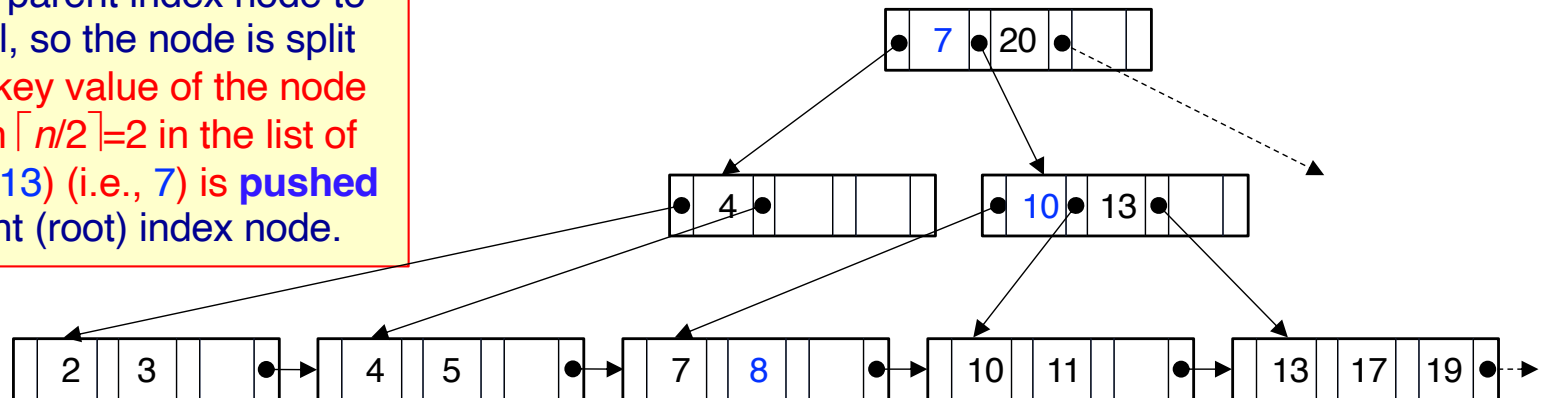
Insert: 8

The leaf node is split and the search-key value of the node entry at position $\lceil n/2 \rceil + 1 = 3$ of the list of values (7, 8, 10, 11) (i.e., 10) is **copied up** to the parent index node.



B⁺-tree before insertion of 8.

This causes the parent index node to become overfull, so the node is split and the search-key value of the node entry at position $\lceil n/2 \rceil = 2$ in the list of values (4, 7, 10, 13) (i.e., 7) is **pushed up** to the parent (root) index node.



B⁺-tree after insertion of 8.

EXERCISE 2

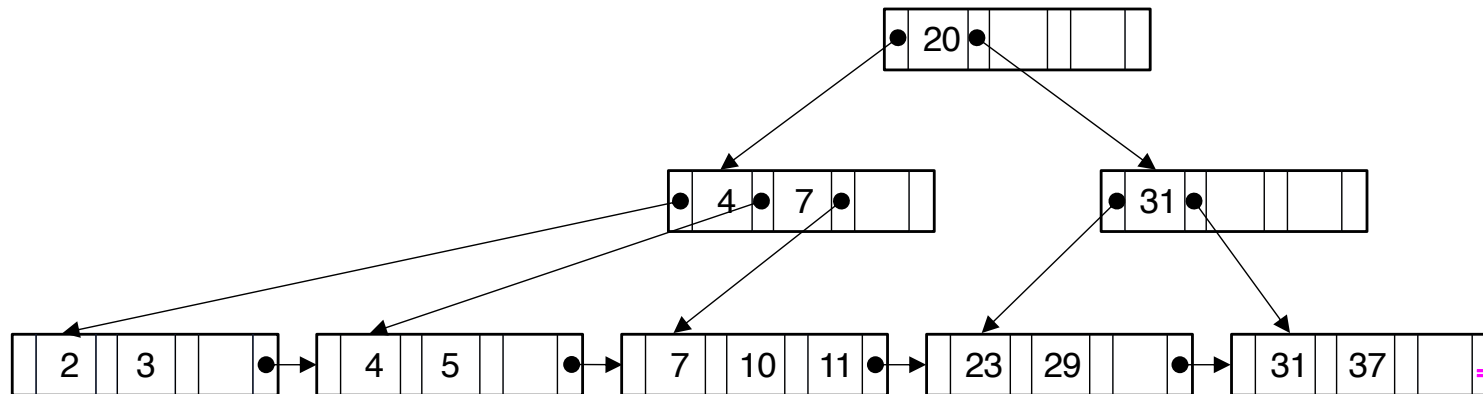
Using initial the B⁺-tree given below with **fan out 4** and the B⁺-tree insertion and deletion algorithms given in the lecture notes, show the B⁺-tree that would result **after** performing the following operations **in the order given**.

i. delete 5

ii. delete 3

iii. delete 7

iv. delete 11



Non-leaf nodes: $\min \lceil 4/2 \rceil = 2$ pointers; $\min \lceil 4/2 \rceil - 1 = 1$ value

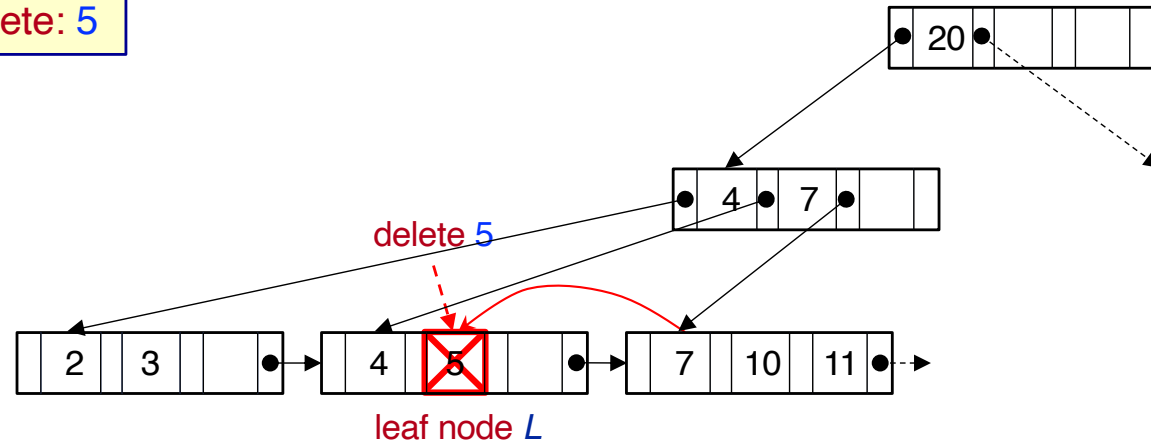
Leaf nodes: $\min \lceil (4-1)/2 \rceil + 1 = 3$ pointers; $\min \lceil (4-1)/2 \rceil = 2$ values

EXERCISE 2 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

The deletion causes the second leaf node to become **underfull** (less than $\lceil (n-1)/2 \rceil = 2$ values).

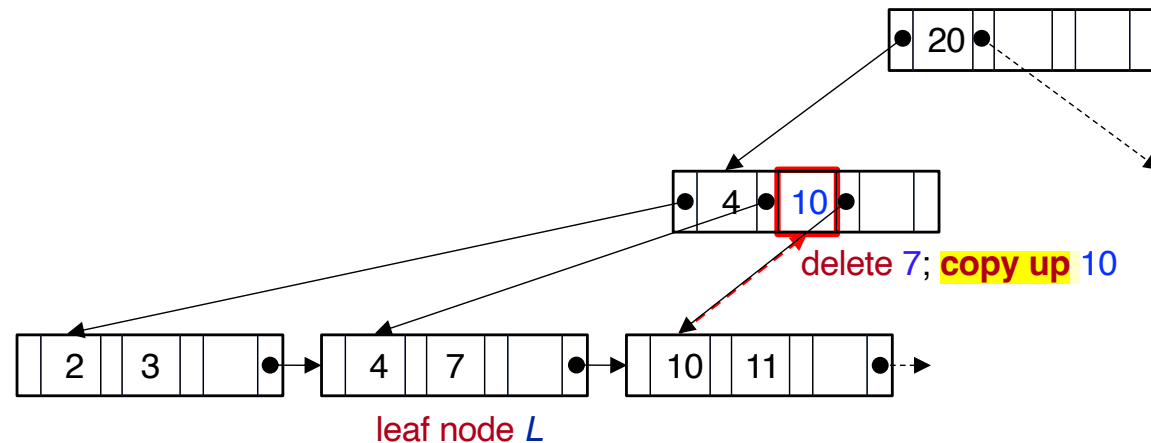
Delete: 5



B⁺-tree before deletion of 5.

The node can borrow a value (7) from its right sibling.

The parent index node is **adjusted** accordingly by deleting 7 and **copying up** 10.



B⁺-tree after deletion of 5.

EXERCISE 2 (CONT'D)

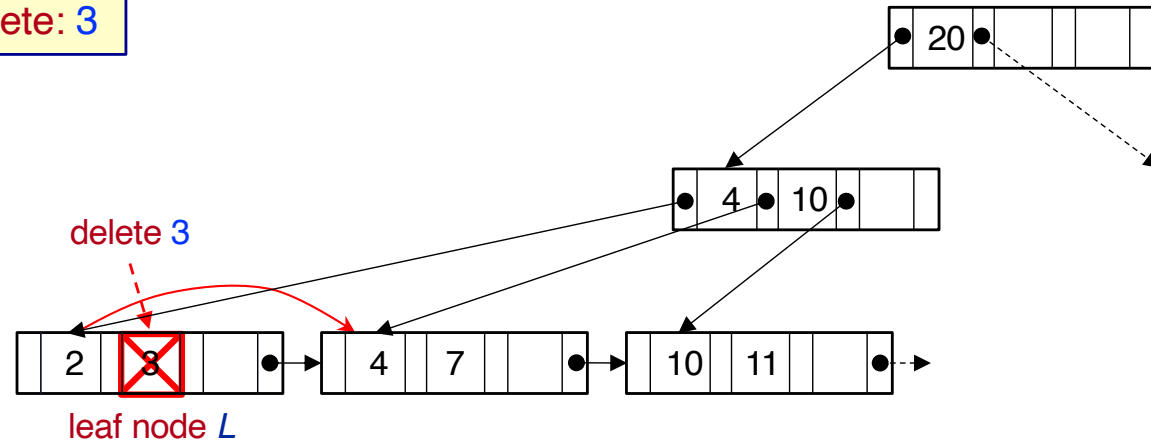
$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

The deletion causes the first leaf node to become **underfull** (less than $\lceil (n-1)/2 \rceil = 2$ values).

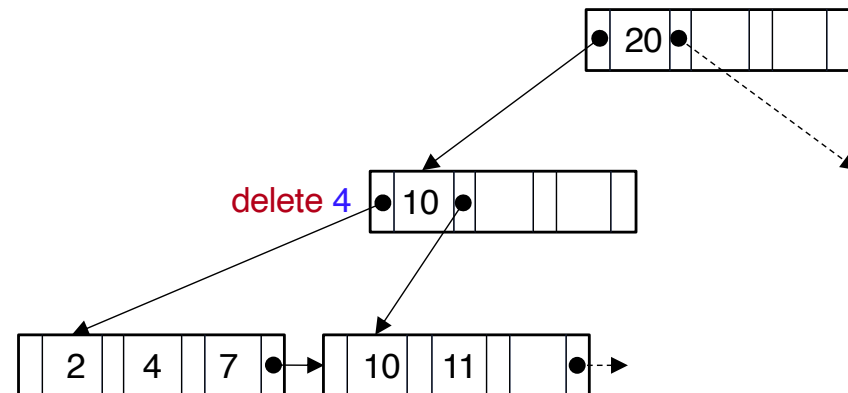
The node cannot borrow a value from its right sibling, so it **must be merged** with it.

The parent **index node** is **adjusted** accordingly by deleting 4.

Delete: 3



B⁺-tree before deletion of 3.



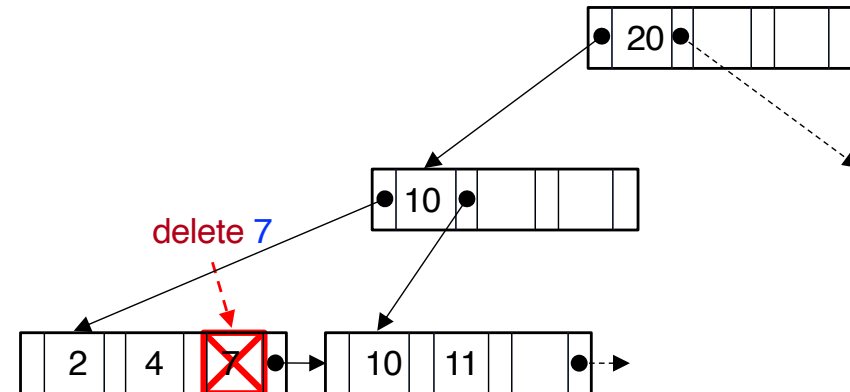
B⁺-tree after deletion of 3.

EXERCISE 2 (CONT'D)

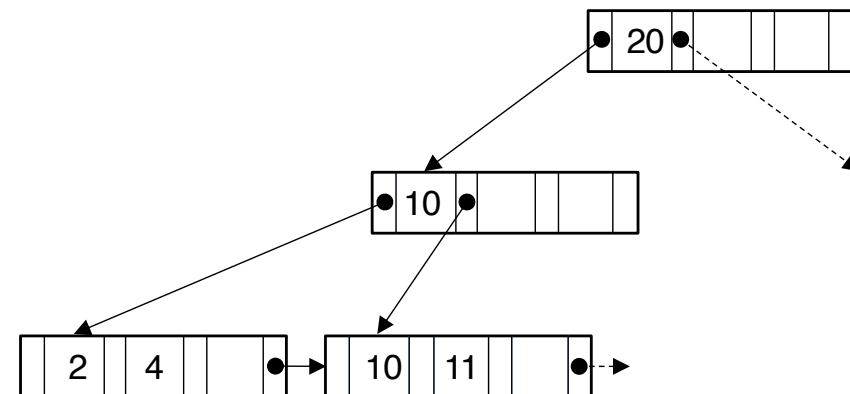
The value is deleted
in the first leaf node.

Delete: 7

$n = 4$ (fan out of B⁺-tree node)
non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)



B⁺-tree before deletion of 7.



B⁺-tree after deletion of 7.

EXERCISE 2 (CONT'D)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

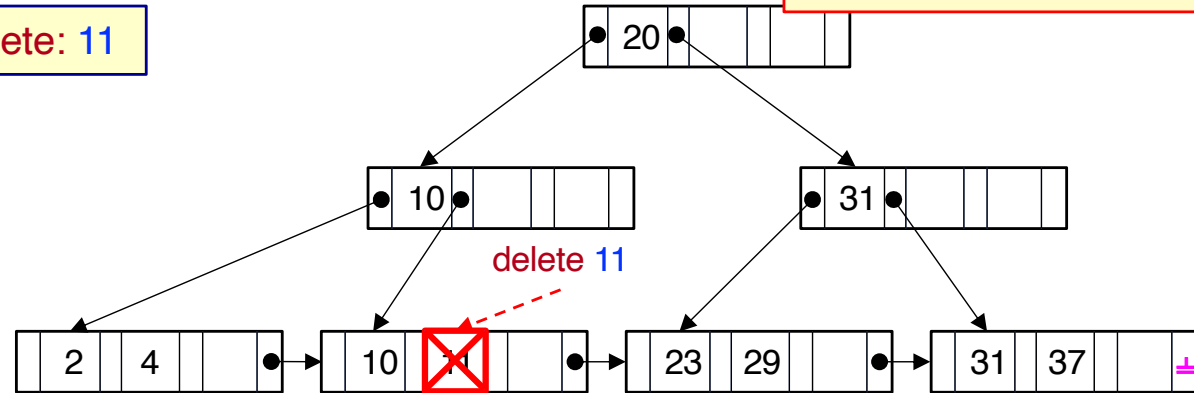
The deletion causes the second leaf node to become **underfull** (less than $\lceil (n-1)/2 \rceil = 2$ values).

The node cannot borrow a value from either of its siblings, so it **must be merged** (select left sibling).

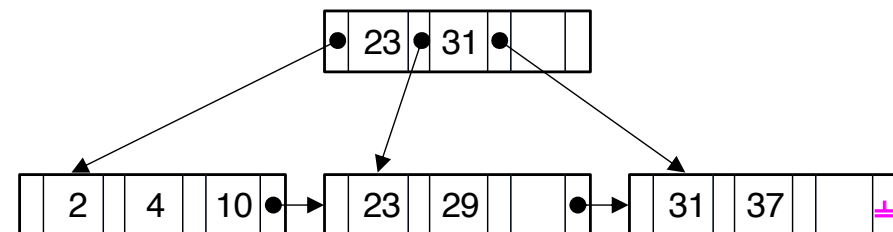
This causes the parent index node to now have only 1 pointer, but it needs 2. Therefore, it **must be merged** with its sibling and the index values adjusted.

This merge causes the root index node to now have only 1 pointer, so it can be deleted, and **the tree shrinks one level**.

Delete: 11



leaf node *L*
B⁺-tree before deletion of 11.



B⁺-tree after deletion of 11.

EXERCISE 3

A company database has the following file and sizes of each field

Employee(employeeId: 6 bytes; employeeName: 10 bytes; departmentId: 4 bytes)

where departmentId is the id of the department where the employee works.

There are 100,000 employee records and 1,000 departments (assume that each department has exactly 100 employees).

A page is 1,000 bytes and a pointer is 4 bytes.

Assume that the file is sorted on departmentId and there is no index.

We want to build a hash index on employeeId on the above file where each entry has the form <employeeId, pointer>.

EXERCISE 3 (CONT'D)

a) How many index entries are needed? (Briefly explain your answer.)

Index entries needed: 100,000 Why?

Explanation: Since a hash index is always secondary, it must be dense (i.e., one entry for each record in the file).

b) How many pages, $B_{\text{employeeIdIndex}}$, are required for these index entries (assuming full pages)?

$bf_{\text{employeeIdIndex}}: \lfloor 1000 / 10 \text{ bytes per entry} \rfloor = \underline{100}$ entries per page

$B_{\text{employeeIdIndex}}: \lceil 100,000 \text{ index entries} / 100 \text{ index entries per page} \rceil = \underline{1000}$

c) Using the hash index, what is the page I/O cost of retrieving the record of an employee with a given employeeId, assuming that there are **no overflow pages**? (Briefly explain your answer.)

Page I/O cost: 2 Why?

Explanation: 1 access to the hash index + 1 access to retrieve the record.

EXERCISE 4

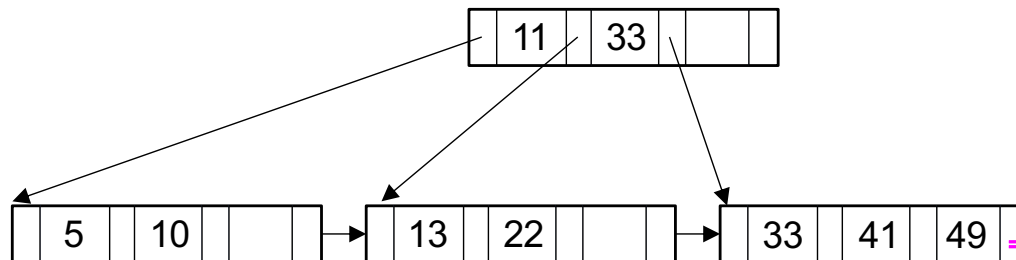
Using initial the B⁺-tree given below with **fan out 4** and the B⁺-tree insertion and deletion algorithms given in the lecture notes, show the B⁺-tree that would result **after** performing the following operations **in the order given**.

i. insert 45

ii. insert 35

iii. insert 40

iv. delete 49

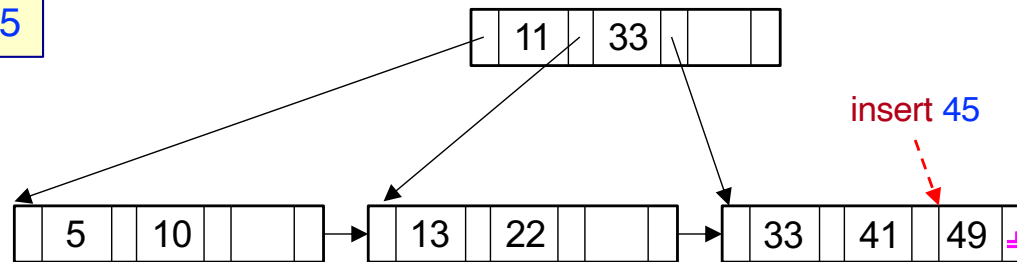


Non-leaf nodes: $\min \lceil 4/2 \rceil = 2$ pointers; $\min \lceil 4/2 \rceil - 1 = 1$ value

Leaf nodes: $\min \lceil (4-1)/2 \rceil + 1 = 3$ pointers; $\min \lceil (4-1)/2 \rceil = 2$ values

EXERCISE 4 (CONT'D)

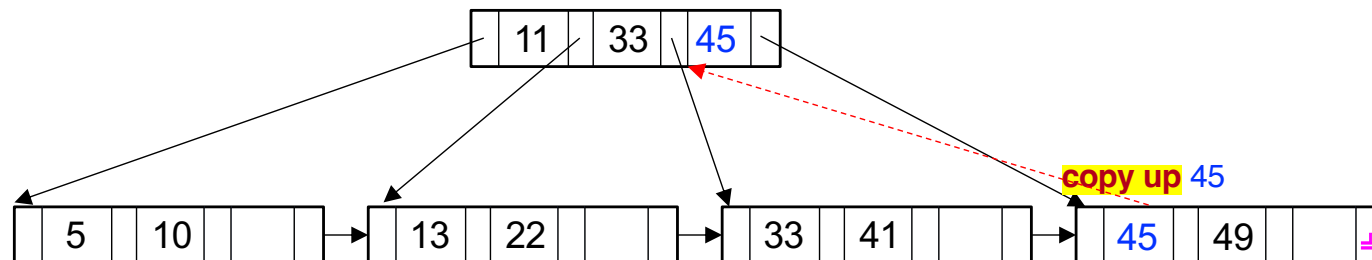
Insert: 45



The value is **inserted** in the right-most leaf node.

This causes the node to **become overfull and split**.

The search-key value at position $\lceil n/2 \rceil + 1 = 3$ (i.e., 45) in the list of values (33, 40, 45, 49) is **copied up** to the parent index node.

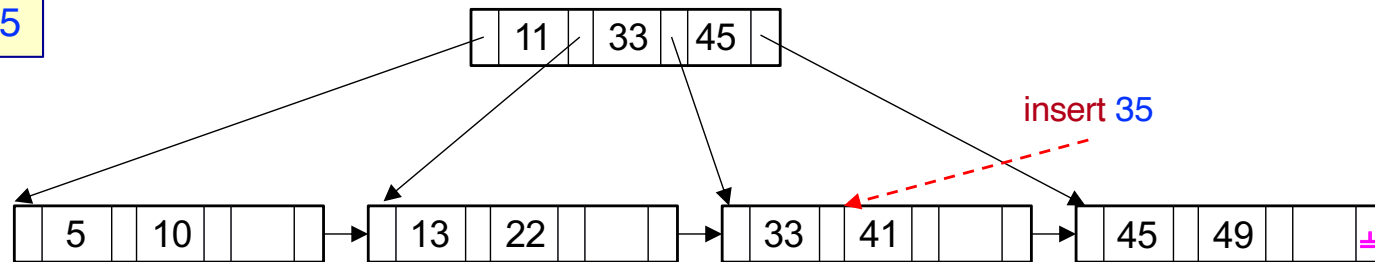


$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

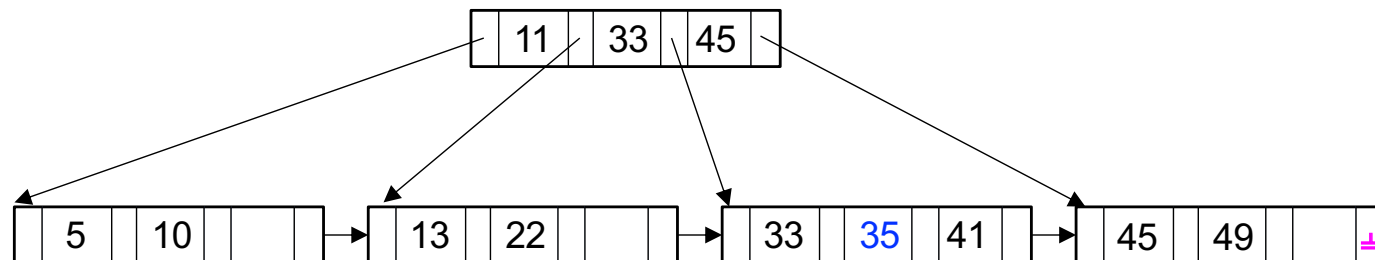


EXERCISE 4 (CONTD)

Insert: 35



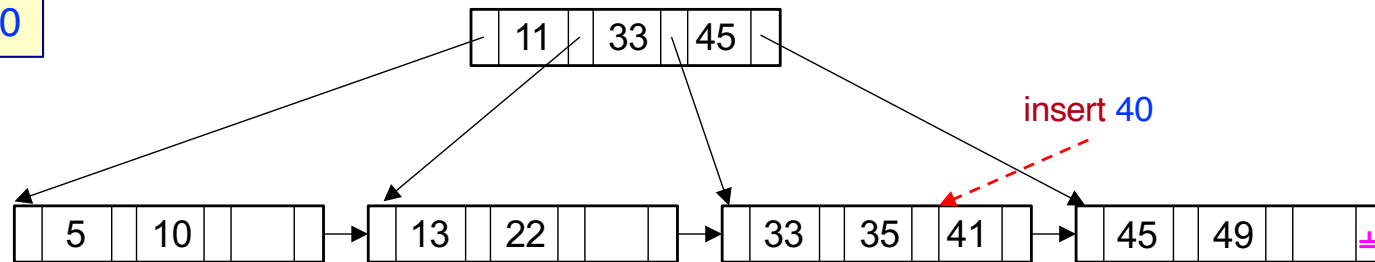
The value is inserted in the **third leaf node** from the left in order.



$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

EXERCISE 4 (CONT'D)

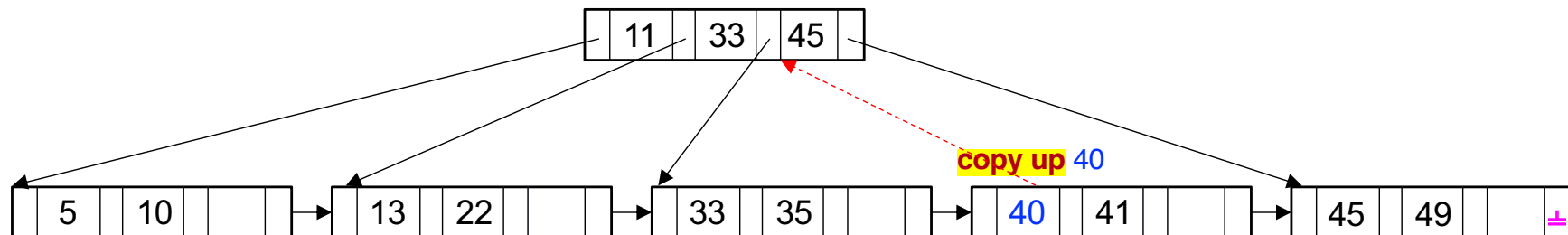
Insert: 40



The value is inserted in the **third leaf node** from the left.

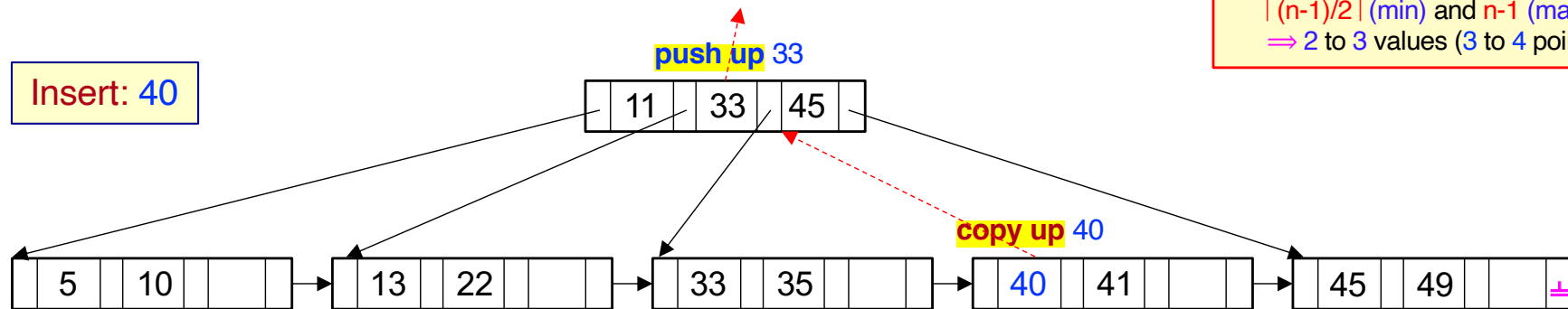
This causes the node to become **overfull** and **split**.

The search-key value at position $\lceil n/2 \rceil + 1 = 3$ (i.e., 40) in the list of values (33, 35, 40, 41) is **copied up** to the parent index node.



EXERCISE 4 (CONTD)

Insert: 40

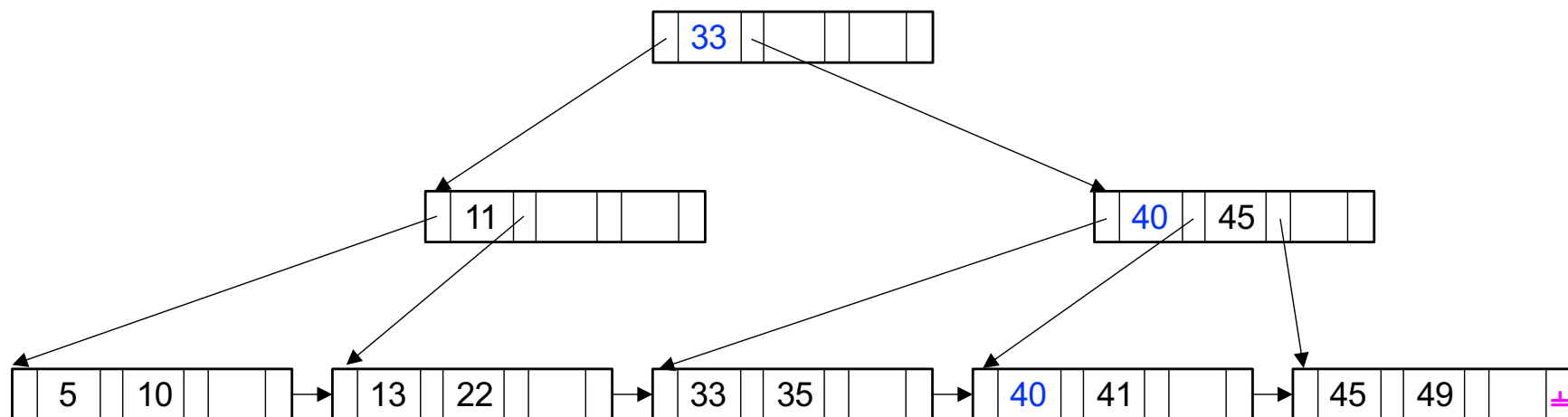


$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

This causes the parent node to become **overfull** and **split**.

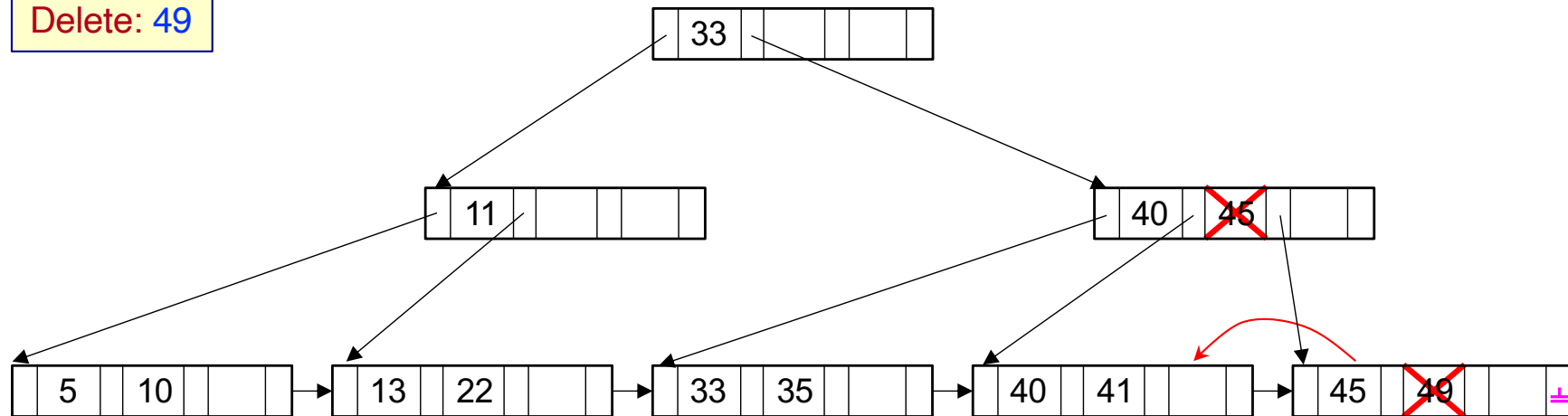
The first $\lceil n/2 \rceil - 1 = 1$ value is placed in the existing node. The search-key value at position $\lceil n/2 \rceil = 2$ (i.e., 33) in the list of values (11, 33, 40, 45) is **pushed up** into the new root node.

The remaining 2 values are placed in a **new internal node**.



EXERCISE 4 (CONTD)

Delete: 49



$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values
 $\Rightarrow 1$ to 3 values (2 to 4 pointers)
 leaf nodes:
 $\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values
 $\Rightarrow 2$ to 3 values (3 to 4 pointers)

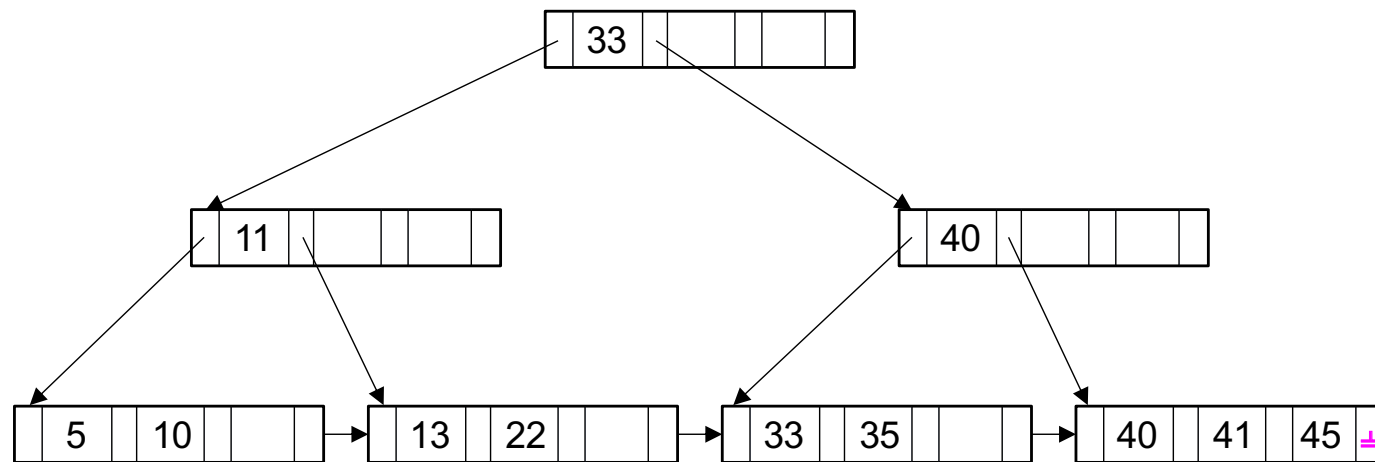
Deleting 49 causes the right-most leaf node to become underfull (i.e., it has less than 2 values).

Since a value cannot be borrowed from its right sibling, the left-most leaf node is merged with its right sibling.

The parent node is adjusted by deleting 45.

EXERCISE 4 (CONTD)

Final B⁺-tree



EXERCISE 5

We want to construct a B⁺-tree for the primary key of an ordered, sequential file of 1,000,000 records, each 25 bytes, using **bulk loading** so that all B⁺-tree nodes have **minimum occupancy** *where possible*.

The primary key occupies 4 bytes.

All pointers are 4 bytes.

The page size is 128 bytes.

a) What is the blocking factor, $bf_{dataFile}$, of the data file?

$$bf_{dataFile}: \lfloor 128 / 25 \rfloor = \lfloor 5.12 \rfloor = \underline{5} \text{ records/page}$$

b) How many pages, $B_{dataFile}$, are needed for the data file?

$$B_{dataFile}: \lceil 1,000,000 / 5 \rceil = \lceil 200,000 \rceil = \underline{200,000} \text{ pages}$$

EXERCISE 5 (CONTD)

search key size: 4 bytes
 pointer size: 4 bytes
 non-leaf internal pages (nodes):
 $\lceil n/2 \rceil - 1$ (min) and $n-1$ (max) values
 leaf pages (nodes):
 $\lceil (n-1)/2 \rceil$ (min) and $n-1$ (max) values

c) What is the maximum number of values in each page of the B⁺-tree index?

~~Maximum number of values: $\lfloor 128 / (4+4) \rfloor = \lfloor 16 \rfloor = 16$~~ Is this correct? **X**

B⁺-tree page structure:

P_1	K_1	P_2	K_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-------	-----	-----------	-----------	-------

For both the leaf and internal nodes of a B⁺-tree there is one more pointer than values. Consequently, for a page size of 128 bytes there are at most $(128-4)=124$ bytes available to store (*pointer*, value) index entries (i.e., to store values).

Maximum number of values: $\lfloor (128 - 4) / 8 \rfloor = \lfloor 15.5 \rfloor = \underline{15}$

d) What is the fan-out, n , of the B⁺-tree?

Fan-out n : 16 (one more than the number of values)

e) What is the minimum occupancy (values) for the B⁺-tree internal pages (nodes)?

Minimum internal node occupancy: $\lceil n / 2 \rceil - 1 = \lceil 16 / 2 \rceil - 1 = \underline{7}$ values (8 pointers)

f) What is the minimum occupancy (values) for the B⁺-tree leaf pages (nodes)?

Minimum leaf node occupancy: $\lceil (n - 1) / 2 \rceil = \lceil 15 / 2 \rceil = \lceil 7.5 \rceil = \underline{8}$ values (9 pointers)

EXERCISE 5 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil 16/2 \rceil - 1$ (min) and $16 - 1$ (max) values
 $\Rightarrow 7$ to 15 values (8 to 16 pointers)
 leaf nodes:
 $\lceil (16-1)/2 \rceil$ (min) and $16 - 1$ (max) values
 $\Rightarrow 8$ to 15 values (9 to 16 pointers)

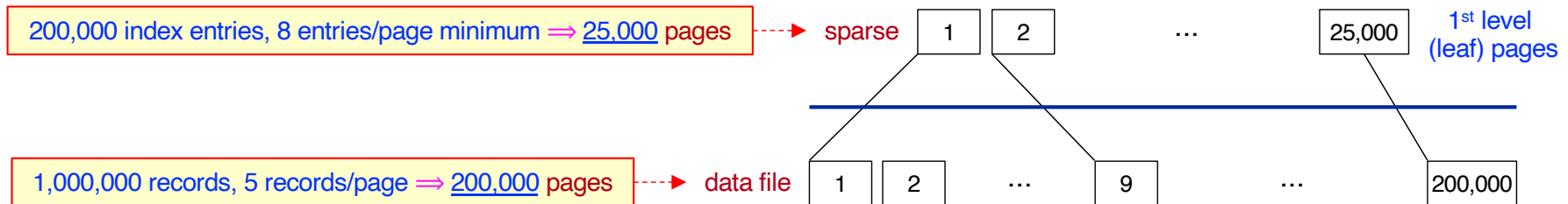
- g) What is the height of the B⁺-tree index and how many pages are there at each level of the B⁺-tree?

Since the data file is an ordered file, the leaf pages of the B⁺-tree will need to **point to only each page of the data file** of which there are 200,000. Thus, the index is sparse.

Each leaf page of the B⁺-tree points to a minimum of 8 data file pages. Note that the last pointer in a leaf page points to the next leaf page and not to a data file page.

Thus, the effective fan-out of the leaf pages is a minimum of 8.

Number of 1st level (leaf) pages: $\lfloor 200,000 / 8 \rfloor = \lfloor 25,000 \rfloor = \underline{25,000}$



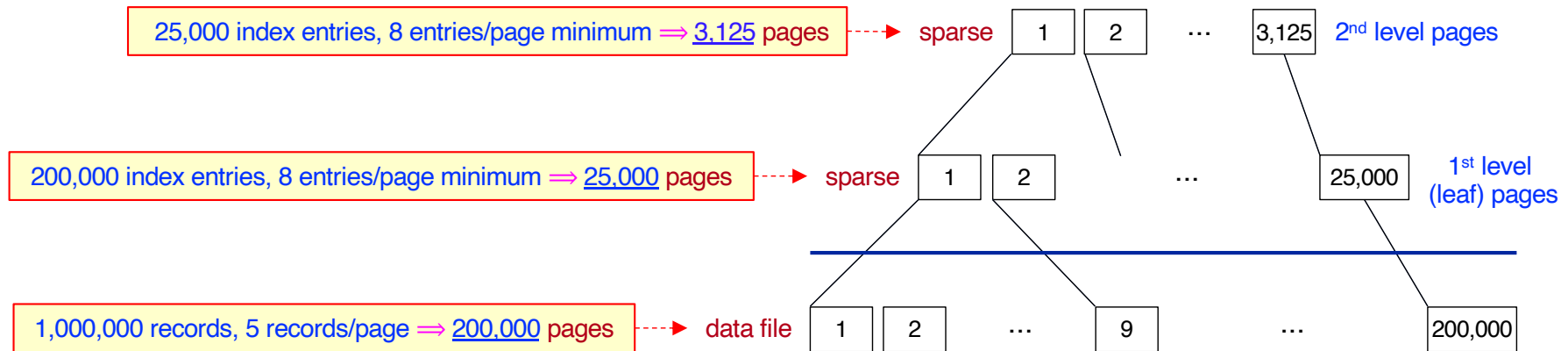
For the given B⁺-tree parameters, all the leaf pages will hold exactly the minimum number of values (i.e., $25,000 \times 8 = 200,000$).

EXERCISE 5 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil 16/2 \rceil - 1$ (min) and $16 - 1$ (max) values
 $\Rightarrow 7$ to 15 values (8 to 16 pointers)
 leaf nodes:
 $\lceil (16 - 1)/2 \rceil$ (min) and $16 - 1$ (max) values
 $\Rightarrow 8$ to 15 values (9 to 16 pointers)

At the 2nd level we need to point to 25,000 1st level (leaf) pages.

Number of 2nd level pages: $\lfloor 25,000 / 8 \rfloor = \lfloor 3,125 \rfloor = \underline{3,125}$



For the given B⁺-tree parameters, all the 2nd level pages will hold exactly the minimum number of values (i.e., $3,125 \times 8 = 25,000$).

Continuing in this manner we arrive at the following.

Number of 3rd level pages: $\lfloor 3,125 / 8 \rfloor = \lfloor 390.63 \rfloor = \underline{390}$

Number of 4th level pages: $\lfloor 390 / 8 \rfloor = \lfloor 48.75 \rfloor = \underline{48}$

Number of 5th level pages: $\lfloor 48 / 8 \rfloor = \lfloor 6 \rfloor = \underline{6}$

Number of 6th level (root) pages: $\lfloor 6 / 8 \rfloor = \lfloor 0.75 \rfloor = \underline{1}$

B⁺-tree height: 6

EXERCISE 5 (CONTD)

$n = 4$ (fan out of B⁺-tree node)
 non-leaf (internal) nodes:
 $\lceil 16/2 \rceil - 1$ (min) and $16 - 1$ (max) values
 $\Rightarrow 7$ to 15 values (8 to 16 pointers)
 leaf nodes:
 $\lceil (16 - 1)/2 \rceil$ (min) and $16 - 1$ (max) values
 $\Rightarrow 8$ to 15 values (9 to 16 pointers)

Why is the floor and not the ceiling function used in the calculations?

If we use the ceiling function, then, for the 3rd level, the number of pages would be $\lceil 3,125 / 8 \rceil = \lceil 390.63 \rceil = 391$.

In this case, the last node would hold only $3,125 - (390 \cdot 8) = 3,125 - 3,120 = 5$ pointers (4 values), which is less than the minimum required occupancy. Thus, when using the floor function, every B⁺-tree node at this level, except the last one, would hold 8 pointers (7 values) while the last one would hold $8 + 5 = 13$ pointers (12 values), maintaining the minimum occupancy requirement.

For the root node, however, the ceiling function is used since the root node is not required to follow the minimum occupancy requirement, and we need to point to the nodes at the next level.

Consequently, we have the following occupancy for the last node at each level:

	<u># pages</u>	<u>last node occupancy</u>
1 st level (leaf) :	25,000	8 pointers (7 values)
2 nd level:	3,125	8 pointers (7 values)
3 rd level:	390	13 pointers (12 values)
4 th level:	48	14 pointers (13 values)
5 th level:	6	8 pointers (7 values)
6 th level (root) :	1	6 pointers (5 values)

EXERCISE 6

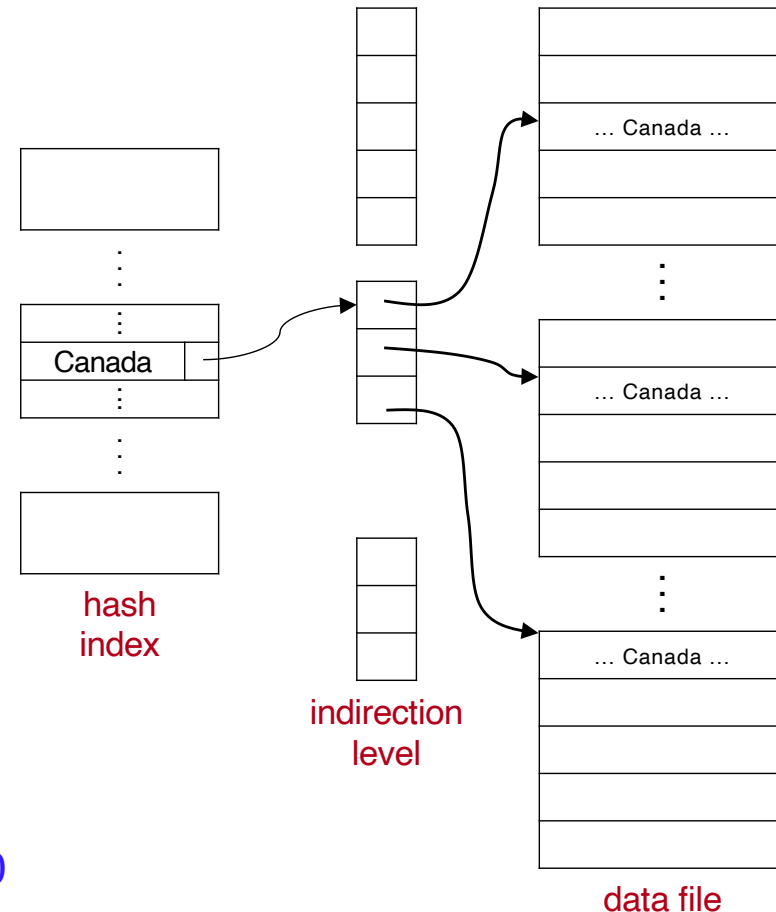
A global e-commerce website maintains a Customer file with attributes customerId, name, address, email and country. The file is organized as a hash file on customerId. There is also a secondary hash index on country. For each country there is only one index entry in the hash index. Assume that on average there are 12,000 customers for each country, there are 120 countries and that a page can hold 120 record pointers. How many page I/Os are required to retrieve the records of all the customers for a given country using the hash index on country?

Explain your calculations; don't just give numbers.

EXERCISE 6 (CONT'D)

From the problem description, the secondary hash index must use a level of indirection to store all the record pointers to the records for a given country as shown in the figure.

Consequently, the number of page I/Os required to retrieve the records for all the customers in each county is:



Hash index page I/Os: 1

Indirection level page I/Os: $\lceil 12,000 / 120 \rceil = 100$

Record retrieval page I/Os: 12,000 (one for each indirection pointer)

Total page I/Os: $1 + 100 + 12,000 = \underline{12,101}$